ST ALOYSIUS
(DEEMED TO BE UNIVERSITY)
MANGALURU 575 003 - INDIA
SCHOOL OF ENGINEERING

Multi-Process Microcontroller Simulator: Design, Implementation & Evaluation of STC89C52 Microcontroller

Register Number	Student Name
25190111	CYRUS SHOBITH PINTO
25190110	CHRISEL LOBO
25190106	ARNOLD NOAH KARYAPPA
25190148	SHASHIDHARA POOJARI

Department of Computer Science & Engineering

Project-Based Learning (PBL) Report

ABSTRACT

This report presents the design, implementation, and empirical evaluation of an educational, instruction-accurate multi-process software simulator for a 19-opcode subset of the STC89C52 (Intel MCS-51) 8-bit microcontroller [1], [2]. Many educational microcontroller simulators use single-threaded monolithic architectures where user interface rendering or synchronous disk logging can stall the simulated CPU execution loop [3]. To eliminate this coupling, we engineered a three-process system communicating over local TCP loopback sockets: **CoreProcess** (CPU Fetch-Decode-Execute engine, 8 KB code ROM, 256-byte internal RAM, hardware LIFO stack, and 16-slot circular FIFO queue on port 5000), **LoggerProcess** (append-only execution log daemon writing to `simulator.log` on port 5001), and **UIProcess** (HTTP web server and 7-panel interactive dashboard on port 8080). To guarantee that a stalled downstream logger never blocks CPU stepping when OS socket buffers fill, CoreProcess decouples log dispatch through a bounded 1,024-slot non-blocking `ArrayBlockingQueue` with an atomic drop counter (explicitly trading audit log completeness under extreme backpressure for deterministic CPU execution). Additionally, we implemented the 3GPP TS 36.212 CRC-24b error-detection polynomial (g(D) = 0x800063) [5] as a Java hardware-model routine operating on simulated 8-bit registers R5, R6, and R7 and the PSW carry flag (CY). Across 38 automated test assertions—including a 20-item circular FIFO wrap-around test, a `CountDownLatch` stalled-logger CPU step test, and an exhaustive sweep of all 96 single-bit and 4,560 two-bit codeword error patterns—the system achieved 100% verification pass rates while maintaining sub-millisecond median interactive step latency (0.914 ms p50).

Keywords: Microcontroller Emulation, Multi-Process Architecture, Inter-Process Communication, Fetch-Decode-Execute, Instruction-Accurate Virtualization, CRC-24b Error Detection.

TABLE OF CONTENTS

1. Introduction	**4**
1.1 Problem Statement	4
1.2 Project Objectives	4
2. The Simulator Design and Working	**4**
2.1 Microcontroller Architecture Study	4
2.2 System Architecture and Design Decisions	5
2.3 Operational Workflow and GUI Layout	5
2.4 Incremental Implementation by Week	6
2.5 Multi-Process IPC Architecture & Benchmarking	6
3. Simulation Examples Testing and Screenshots	**7**
3.1 Live 3GPP LTE-B CRC-24b Lab & Simulation Walkthroughs	7
4. Individual Contribution	**8**
4.1 Weekly Task Breakdown by Member	8
4.2 Agreed Team Contribution Summary (30% / 26% / 24% / 20%)	9
4.3 Individual Personal Reflections	9
5. Team Communication	**10**
6. Problems Faced and How They Were Resolved	**11**
7. From Theory to Practice	**11**
7.1 Operating Systems Concepts	11
7.2 Data Structures & Algorithms	11
7.3 Computer Networks	11
7.4 Cross-Disciplinary Synthesis Reflection	11
8. Conclusion, Limitations and Future Work	**12**
References	**12**
Final Report Sign-off & Faculty Mentor Endorsement	**13**

1. Introduction

1.1 Problem Statement

Modern computer engineering pedagogy requires students to understand how hardware state machines, memory segmentation, register banks, and instruction pipelines execute in silicon [1], [4]. However, physical microcontroller kits present hardware maintenance bottlenecks, lack visibility into internal CPU registers during single-cycle stepping, and restrict collaborative debugging. Existing educational 8051 simulators [3] typically rely on monolithic, single-threaded architectures where UI rendering or synchronous disk logging can stall the simulated CPU loop. Furthermore, implementing 24-bit telecommunications error-detection algorithms—such as the 3GPP TS 36.212 LTE CRC-24b standard [5]—across an 8-bit register file requires explicit multi-byte register cascading (R5:R6:R7) and bit-serial GF(2) division through the PSW carry flag.

1.2 Project Objectives

1. Virtualize a 19-opcode educational subset of the STC89C52/8051 architecture [2] in Java with instruction-accurate Fetch-Decode-Execute stages, 4 register banks (R0–R7), and PSW flags (CY, AC, OV, P).
2. Engineer a decoupled three-process architecture over local TCP loopback sockets (Ports 5000, 5001, 8080) with a bounded non-blocking `ArrayBlockingQueue(1024)` log dispatcher prioritizing CPU determinism.
3. Construct a 7-panel interactive Web Dashboard (Port 8080, Figure 3) featuring live instruction disassembly, Fetch-Decode-Execute pipeline trace, hardware LIFO stack visualization, and 16-slot circular FIFO queue inspection.
4. Build an independent append-only logging daemon (`LoggerProcess` on Port 5001) persisting timestamped execution and CRC telemetry traces to `simulator.log`.
5. Implement and benchmark the 3GPP TS 36.212 CRC-24b algorithm (g(D) = 0x800063) [5] as a Java hardware-model routine (`step8051Hardware()`) operating on simulated registers R5:R6:R7 (Figure 4), verifying 100% detection across all 96 single-bit and 4,560 two-bit codeword error patterns.

2. The Simulator Design and Working

2.1 Microcontroller Architecture Study

The target MCU (STC89C52RC / Intel MCS-51 family [2]) possesses a segmented memory and register hierarchy mapped within our 19-opcode Java virtualization environment as detailed in Table 1.

Table 1: STC89C52 / MCS-51 Hardware Specification [2] and Simulator Virtualization Design

Hardware Feature	MCS-51 Hardware Specification [2]	Simulator Virtualization Design
Data Word Size	8-bit ALU & Datapath	Explicit bitwise masking (& 0xFF) on all register and RAM reads/writes in Java to prevent 32-bit signed integer sign-extension.
Program Counter (PC)	16-bit addressing (8 KB On-Chip Flash ROM)	16-bit unsigned index into an 8,192-byte code ROM array (Memory.java); halts on synthetic 0xFF or standard 8051 SJMP $ (80 FE) self-loop.
Internal Data RAM	256 Bytes on-chip RAM (00H – FFH)	Partitioned into Register Banks 0–3 (00H–1FH), Bit-Addressable space (20H–2FH), and General Scratchpad RAM (30H–7FH).
Special Function Regs	Direct addressing (80H – FFH)	Mapped SFRs: Accumulator ACC (0xE0), B (0xF0), PSW (0xD0), Stack Pointer SP (0x81), and I/O Ports P0–P3 (0x80, 0x90, 0xA0, 0xB0).
ALU Status Flags	Program Status Word (PSW, 0xD0)	Boolean status register (PSW.java) computing Carry (CY), Auxiliary Carry (AC), Overflow (OV), Register Bank Select (RS1:RS0), and even Parity (P).
Stack & FIFO Queue	Hardware LIFO Stack + 16-Slot Buffer	Pre-increment PUSH / post-decrement POP starting at SP=07H (with underflow guard) plus a 16-slot circular FIFO ring buffer (DataStructures.java).

2.2 System Architecture and Design Decisions

To eliminate single-process coupling [3], the simulator executes as three independent operating system processes communicating over local TCP loopback sockets (`127.0.0.1`, Figure 1): (1) **CoreProcess** (`Port 5000`, Chrisel & Cyrus) hosts the 19-opcode Fetch-Decode-Execute CPU engine (`CPU.java`), 8 KB ROM, 256-byte RAM, hardware stack, 16-slot circular FIFO queue, and the 3GPP CRC-24b engine (`CRC24b.java`); (2) **LoggerProcess** (`Port 5001`, Arnold) runs a dedicated `ServerSocket` daemon parsing `LOG|LEVEL|CAT|MSG` frames and persisting them to `simulator.log` via `synchronized LogHandler.writeLog()`; and (3) **UIProcess** (`Port 8080`, Shashidhara) hosts an embedded HTTP web server serving the 7-panel Web Dashboard (`index.html`) alongside a CLI controller and Swing desktop client. Crucially, if `LoggerProcess` stalls and the OS TCP socket buffer fills, a synchronous socket write would block the CPU thread. To prevent this, `CoreProcess` enqueues all outbound log frames into a bounded 1,024-slot `ArrayBlockingQueue` using non-blocking `offer()` with an atomic `droppedLogs` counter and a background daemon drain thread—explicitly prioritizing deterministic CPU instruction stepping over audit log completeness under severe I/O backpressure.

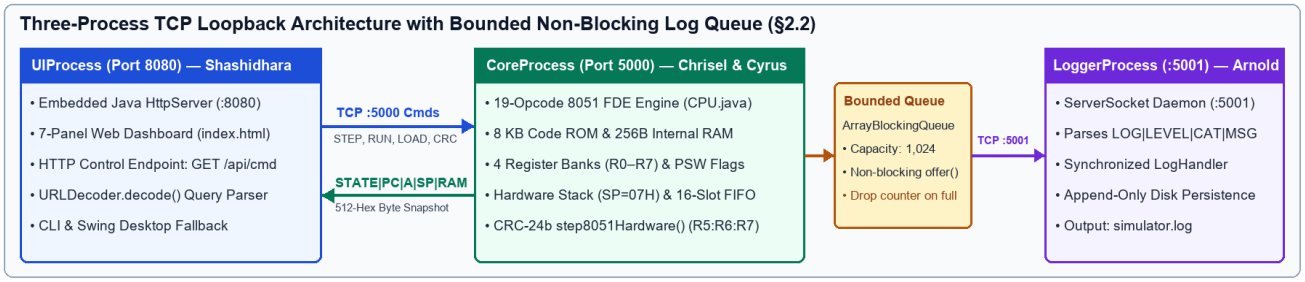


Figure 1: Three-Process TCP Loopback Architecture with Bounded 1,024-Slot Non-Blocking Log Queue (§2.2).

2.3 Operational Workflow and GUI Layout

As illustrated in the Figure 2 workflow flowchart and Figure 3 live screenshot, the Web Dashboard at `http://localhost:8080/` organizes silicon telemetry into 7 panels: **(1) Program Memory & Disassembly** with editable hex bytecode, 5 one-click presets, and live `PC` pointer; **(2) CPU Pipeline Trace** displaying active Fetch, Decode, and Execute stages; **(3) Working Registers & PSW Flags** (`PC, SP, ACC, B, R0–R7, CY, AC, OV, P`); **(4) Hardware Stack Tower** tracking `SP` growth from `0x07`; **(5) 16-Slot Circular FIFO Queue Grid** showing `FRONT` and `REAR` pointers; **(6) 256-Byte Internal RAM Hex Matrix**; and **(7) the 3GPP LTE-B CRC-24b Lab** (Figure 4). When a user clicks Step, Run, Preset, or CRC in the browser, `GET /api/cmd?cmd=...` is decoded via `URLDecoder.decode()` in `UIProcess` and transmitted over TCP Port 5000 to `CoreProcess`. CoreProcess acquires `cpuLock`, executes the step, returns a pipe-delimited `STATE|...` snapshot to refresh all 7 browser panels, and asynchronously enqueues an execution log frame via `offer()`.

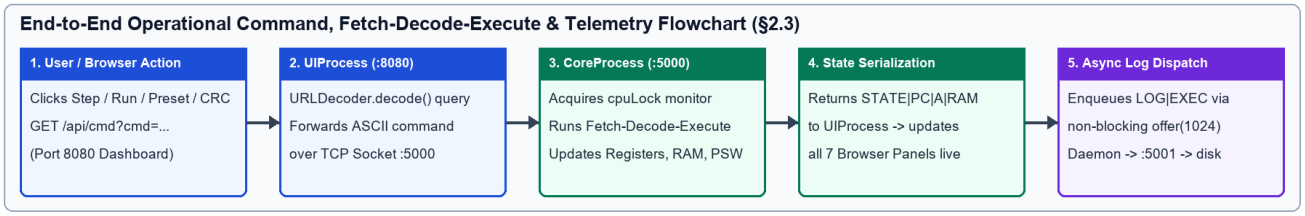


Figure 2: End-to-End Operational Command, Fetch-Decode-Execute & Telemetry Flowchart (§2.3).

2.4 Incremental Implementation by Week

Table 2: Five-Week Incremental Implementation Schedule and Deliverables [1]

Sprint / Week	Primary Module Focus	Concrete Functional Deliverables
Week 1	Architecture Modeling & Tooling	STC89C52 datasheet study (10 hardware topics partitioned on Miro), 8051 register & 8 KB ROM / 256B RAM map, Java OOP class design, and Git workflow.
Week 2	Core ALU & Instruction Subset	Fetch-Decode-Execute pipeline in CPU.java, 19 opcodes (NOP, MOV, ADD, SUBB, MUL, DIV, INC, DEC, ANL, ORL, XRL, JZ, JNZ, SJMP, PUSH, ENQUEUE, DEQUEUE, HALT), PSW flags.
Week 3	Data Structures & Memory	Hardware LIFO stack pointer (SP=07H reset, underflow guard at 0x07), 16-slot circular FIFO ring buffer ((rear+1)%16), and QueueTest.java verification.
Week 4	Multi-Process Socket IPC	Decoupled CoreProcess (Port 5000), LoggerProcess (Port 5001), UIProcess (Port 8080) over TCP loopback sockets + bounded 1,024-slot non-blocking log queue.
Week 5	Telecom Extension & Benchmarks	3GPP TS 36.212 CRC-24b Java routine (step8051Hardware on R5:R6:R7), 4,656-case exhaustive error sweep, BenchmarkCRC.java latency audit, and final report.

2.5 Multi-Process IPC Architecture & Benchmarking

To evaluate the performance trade-off between a monolithic single-JVM loop and our decoupled three-process TCP loopback architecture, `BenchmarkCRC.java` was executed on Windows 11 x64 (OpenJDK 26.0.2) using a 49-byte LTE transport block test vector (`"STC89C52-PBL-BONDAAS-..."`). To eliminate cold-start interpreter distortion, the JVM performed **10,000 C2 JIT warm-up iterations** prior to measurement. Table 3 summarizes the empirical comparison across standalone execution, multi-process TCP loopback IPC, and stalled-logger fault-isolation testing.

Table 3: Empirical Benchmark Comparison — Standalone In-Memory vs. Multi-Process TCP Loopback IPC

Evaluation Metric	Standalone In-Memory (Single JVM)	Multi-Process TCP Loopback (:5000 / :5001)
Measured Workload	100,000 CRC-24b packet computations (after 10,000 C2 JIT warm-up iterations)	200 synchronous stop-and-wait TCP command/STATE round-trips on Port 5000
Total Elapsed Time	28.92 ms (cumulative checksum 0x025A3C)	306.92 ms (across 200 full command + 512-char RAM state responses)
Per-Operation Latency	0.0003 ms / packet (0.29 µs)	Mean = 1.535 ms | Min = 0.311 ms | Median (p50) = 0.914 ms | p99 = 10.788 ms
Effective Throughput	3,457,229 packets / sec	~652 interactive state-synchronized steps / sec (at mean RTT; >1,090/sec at p50)
UI Crash Isolation	None — unhandled UI exception terminates JVM	Full OS process isolation — killing UIProcess (:8080) leaves CoreProcess (:5000) intact
Logger Stall Isolation	Synchronous I/O blocks CPU thread	CountDownLatch test: 1,024 queued, 480 dropped via offer(), CPU halts Preset 1 (ACC=2AH)

Performance Analysis & Architectural Trade-offs: Multi-process TCP loopback IPC introduces a median round-trip latency of `0.914 ms (p50)` compared to `0.0003 ms` in-memory execution. This overhead is dominated by two factors: (1) synchronous stop-and-wait request/response socket framing across OS kernel loopback buffers, and (2) serializing all 256 bytes of internal RAM into a 512-character hexadecimal string inside every `STATE|...` response so the browser RAM matrix stays synchronized on every single step. For interactive educational debugging (human stepping at 1–10 Hz or continuous visual animation at 20–60 Hz), a median latency of `0.914 ms` (>1,000 steps/sec capacity) is completely imperceptible to the user while delivering true operating-system process isolation between the UI, CPU Core, and disk Logger.

3. Simulation Examples Testing and Screenshots

Comprehensive automated test suites (**38/38 passing assertions** across `TestCPU`: 20/20, `TestCRC24b`: 9/9, `QueueTest`: 3/3, and `LoggerProcessTest`: 6/6) validated all instruction categories, 20-item circular FIFO wrap-around past slot 16, latch-verified IPC fault isolation, and exhaustive CRC-24b error detection (Table 4, Figures 3 and 4):

Table 4: Automated Verification Matrix With Explicit Source Test Suite Mapping (38/38 Passing)

Row	Operation & Source Test ID	Expected Outcome	Actual Outcome	Status
TR-01	Data Transfer: MOV A,#42H; B,#07H (CPU TC-01)	A=42H, B=07H, PC advances	A=42H, B=07H; PC incremented	PASS
TR-02	ALU Add/Borrow: ADD & SUBB (CPU TC-04, TC-18)	ADD: 01H,CY=1; SUBB: FDH,CY=1	A=01H (CY=1,AC=1); SUBB A=FDH (CY=1)	PASS
TR-03	Control Flow: SJMP, JZ/JNZ, 80 FE (TC-17, TC-19)	JZ/JNZ skip INC; 80 FE halts	INC skipped (A=00H); 80 FE sets isHalt	PASS
TR-04	Stack PUSH ACC (0xC0 0xE0) (CPU TC-15)	SP 07H->08H; RAM[08H]=3CH	SP=08H, RAM[08H]=3CH	PASS
TR-05	FIFO 20-Item Wrap-Around (CPU TC-20, QueueTest)	Fill 16, pop 4, push 4 wraps rear	full=true, wrapped 17..20, dequeued 16	PASS
TR-06	MUL AB (6x7) & DIV AB (B=0) (CPU TC-07, TC-16)	A=2AH, OV=0; DIV B=0->OV=1	A=2AH, OV=false; DIV0 sets OV=true	PASS
TR-07	Latch-Blocked Logger CPU Step (LogTest TC-06)	CPU halts while consumer blocked	ACC=2AH, halted=true (480 queue drops)	PASS
TR-08	Exhaustive CRC-24b Sweep (TestCRC24b TC-01..09)	'123456789'->23EF52; all 1b/2b!=0	96/96 1-bit & 4,560/4,560 2-bit detected	PASS

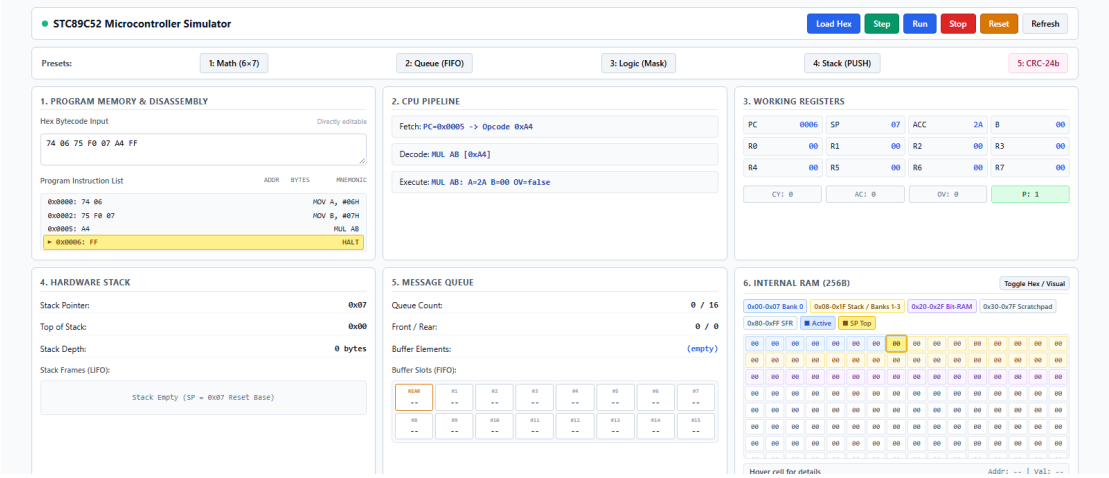


Figure 3: Live STC89C52 Web Dashboard (http://localhost:8080, Panels 1–6) halted at PC=0x0006 on Preset 1 (Math 6x7: 74 06 75 F0 07 A4 FF), showing ACC=2A (42), B=00, P=1, Pipeline MUL AB [0xA4], and 256B Internal RAM.

3.1 Live 3GPP LTE-B CRC-24b Lab & Simulation Walkthroughs



Figure 4: Cropped Panel 7 — 3GPP LTE-B CRC-24b Lab on Web Dashboard verifying "123456789" -> Remainder 0x23EF52 (R5:23 R6:EF R7:52), Clean Syndrome 0x000000, 1-Bit Error (Byte 0 Bit 0) 0x520000, and 2-Bit Burst (Byte 0 Bits 0&1) 0xF60000.

1. Preset 1 Walkthrough — 8-Bit ALU Multiplication & Parity Verification (Figure 3):

Loading bytecode `74 06 75 F0 07 A4 FF` executes four deterministic Fetch-Decode-Execute cycles: (i) `74 06` (`MOV A, #06H`) loads `0x06` into the Accumulator; (ii) `75 F0 07` (`MOV B, #07H`) writes `0x07` into SFR `B` at address `0xF0`; (iii) `A4` (`MUL AB`) multiplies `6 × 7 = 42` (`0x2A`), storing the low byte `0x2A` in `ACC`, clearing `B = 0x00`, clearing `OV = 0` and `CY = 0`, and setting the even-parity flag `P = 1` because `0x2A` (00101010_2) has three set bits; and (iv) `FF` (`HALT`) stops execution at `PC = 0x0006`.

2. Preset 2 & TC-20 Walkthrough — 16-Slot Circular FIFO Queue Wrap-Around:

The embedded message queue in `DataStructures.java` maintains a 16-byte ring buffer with `front`, `rear`, and `size` pointers advancing via `(pointer + 1) % 16`. In `TestCPU.java` (`TC-20`), 16 values (`1..16`) fill the queue to capacity (`isQueueFull() == true`), a 17th push is safely rejected without overwriting unread data, 4 items (`1..4`) are dequeued from `front`, and 4 new items (`17..20`) are enqueued—wrapping the `rear` pointer across the boundary from slot 15 back to slots 0, 1, 2, and 3—before all 16 items (`5..20`) are dequeued in exact FIFO order.

3. 3GPP TS 36.212 CRC-24b Register Cascading & Exhaustive Error Sweep (Figure 4):

To model the 24-bit 3GPP LTE generator polynomial $g(D) = D^{24} + D^{23} + D^6 + D^5 + D + 1$ (`0x800063`) [5] across 8-bit registers, `CRC24b.step8051Hardware()` partitions the 24-bit shift register across Bank 0 registers `R5` (bits 23..16), `R6` (bits 15..8), and `R7` (bits 7..0). For each input byte, the byte is XORed into `R5` and shifted left 8 times across `R7 -> R6 -> R5` through the carry bit; whenever the MSB shifted out of `R5` is 1, the registers are XORed with `0x80`, `0x00`, and `0x63`. For standard payload `"123456789"`, transmitter encoding produces remainder `0x23EF52` (`R5=0x23, R6=0xEF, R7=0x52`) and 12-byte codeword `31..39 23 EF 52`. Receiver verification yields `0x000000` on a clean channel, `0x520000` on a 1-bit flip at byte 0 bit 0, `0xF60000` on a 2-bit burst at byte 0 bits 0&1, and 100% non-zero syndromes across all **96 single-bit** and **4,560 double-bit** error combinations (`TC-09`).

4. Individual Contribution

4.1 Weekly Task Breakdown by Member

Table 5: Weekly Task Allocation Across All Four Team Members

Sprint Week	Cyrus Shobith Pinto (Team Lead)	Chrisel Lobo (Core & FDE Lead)	Shashidhara Poojari (UI & Visuals Lead)	Arnold Noah Karyappa (Logger Lead)
Week 1	Project setup, Miro board, interrupt research, Git repo, task allocation	Timer & Hardware Stack/SP study, 8051 RAM mapping	Memory organization & PC study, Web UI wireframes	CPU registers (R0–R7), PSW flags, GPIO & logger spec
Week 2	Opcode specification, ALU architecture, integration tooling	CPU FDE cycle engine (CPU.java), arithmetic/logic opcodes	CLI terminal display & status monitor prototype	Registers.java & PSW.java support, log format design
Week 3	Data structure specifications, assembly test plan	DataStructures.java: SP stack & 16-slot circular queue	Stack & Circular FIFO visualizer UI layout	Thread-safe synchronized LogHandler.writeLog()
Week 4	Multi-process socket IPC & non-blocking 1,024-slot queue	CoreProcess.java socket server (:5000) & state serializer	UIProcess.java HTTP web server (:8080) & presets	LoggerProcess.java socket daemon (:5001) & file writer
Week 5	CRC-24b step8051Hardware(), BenchmarkCRC.java, final report	Registers R5:R6:R7 mapping, TestCPU (20/20), report	3-Stage CRC-24b Lab UI & real-time noise controls	LoggerProcessTest.java latch test (6/6) & log audit

4.2 Agreed Team Contribution Summary

Contribution shares were agreed upon by all four members based on verifiable repository commit history, subsystem complexity, integration workload, and attendance across the five recorded project meetings (`docs/meetings/`):

Table 6: Agreed Team Member Workstream Distribution and Sign-Off (Backed by Git Commits & Meeting Logs)

Member Name	Student ID / USN	Assigned Workstreams & Verifiable Repository Evidence	Agreed Share (%)	Member Sign-Off
Cyrus Shobith Pinto (Team Leader)	25190111	Multi-Process Architecture, Socket IPC Orchestration, Non-Blocking 1,024-Slot Log Queue, 3GPP CRC-24b Engine, BenchmarkCRC.java, Branch Integration (48 Git commits; 5/5 meetings)	30%	_____
Chrisel Lobo (Core Lead)	25190110	CoreProcess (:5000), 19-Opcode Fetch-Decode-Execute CPU Engine (CPU.java), 256B RAM & Stack/FIFO DataStructures.java, TestCPU (20/20), Documentation (17 Git commits; 5/5 meetings)	26%	_____
Shashidhara Poojari (UI Lead)	25190148	UIProcess HTTP Server (:8080), 7-Panel Interactive Web Dashboard (index.html), SimulatorUI Swing Client, URLDecoder Fix, 5 Silicon Presets (9 Git commits; 5/5 meetings)	24%	_____
Arnold Noah Karyappa (Logger Lead)	25190106	LoggerProcess TCP Daemon (:5001), Thread-Safe Synchronized LogHandler.writeLog(), simulator.log Persistence, LoggerProcessTest (6/6) (20 Git commits; 3/5 meetings)	20%	_____

4.3 Individual Personal Reflections

Reflection by Cyrus Shobith Pinto (Team Leader — 25190111):

Leading the multi-process integration taught me that splitting a simulator across three OS processes solves shared-heap crashes only if the communication boundaries are engineered for backpressure. Early on, I underestimated how easily a slow or stalled socket reader in LoggerProcess could freeze CoreProcess once the OS TCP send buffer filled, and I initially conflated 32-bit Java integer shifts with true 8051 register behavior. Re-architecting CoreProcess around a bounded 1,024-slot ArrayBlockingQueue using non-blocking offer() with an explicit drop counter—and modeling the 3GPP TS 36.212 CRC-24b bit-serial shift across R5:R6:R7 and the carry flag—showed me how to make, test, and document rigorous systems trade-offs.

Reflection by Chrisel Lobo (Core Process Lead — 25190110):

My biggest technical surprise while building CPU.java and the 19-opcode Fetch-Decode-Execute engine was seeing ADD and SUBB produce corrupted carry (CY), auxiliary carry (AC), and overflow (OV) flags whenever operand bytes exceeded 0x7F. Because Java lacks unsigned byte primitives, reading 0xFE from memory automatically sign-extended it to -2 in 32-bit arithmetic. Fixing that bug with explicit (& 0xFF) masks at every register and RAM boundary—alongside guarding stack pop() against underflow below SP = 0x07 and implementing the 8051 hardware rule that DIV AB divide-by-zero sets OV = 1 instead of throwing an exception—taught me how unforgiving hardware emulation is when host-language types leak through.

Reflection by Shashidhara Poojari (UI Subsystem Lead — 25190148):

When I first connected the 7-panel Web Dashboard on Port 8080 to CoreProcess on Port 5000, loading custom hex bytecode strings containing spaces silently failed and caused the disassembly pointer to desynchronize from the Program Counter. Tracing that failure to URL-encoded spaces (%20) on the /api/cmd endpoint and fixing it with URLDecoder.decode() taught me that distributed client-server interfaces almost always fail at serialization boundaries. Designing the live Fetch-Decode-Execute pipeline indicator, the 16-slot circular queue visualizer, and the 3-stage CRC-24b noise-injection controls showed me how visual telemetry makes invisible silicon state transitions immediately understandable.

Reflection by Arnold Noah Karyappa (Logging Subsystem Lead — 25190106):

I initially assumed that opening a ServerSocket on Port 5001 and appending execution lines to simulator.log would be straightforward, until high-frequency instruction bursts risked interleaving log frames and stalling the producer thread. Implementing thread-safe synchronized file appends inside LogHandler.writeLog(), handling malformed frames gracefully without crashing the daemon, and writing the CountDownLatch fault-isolation test in LoggerProcessTest.java taught me the fundamental operating-systems trade-off between strict audit-log completeness and non-blocking producer determinism.

5. Team Communication

Team collaboration was maintained across five structured project phases using synchronous video calls, in-person campus working sessions, daily messaging channels, and disciplined GitHub version control:

- **Google Meet Video Calls (with Live Screen-Sharing & Automated Transcription):** Used for architectural walkthroughs, compiler-to-Java behavior verification, live multi-process socket integration, and final regression testing—specifically Meeting 1 (22 Aug 2026, Meet code `qwi-axoh-wmh`), Meeting 2 (27 Aug 2026 at 4:20 PM, recorded with transcription), Meeting 4 (28 Sep 2026, 3-hour live multi-process IPC coding session), and Meeting 5 (3 Nov 2026, 10:00 PM–11:52 PM IST, Meet code `hbo-ojhk-fzg`).
- **In-Person Campus Working Sessions:** Held at the **College Library** (Meeting 3 on 3 Sep 2026, 1:00 PM–3:30 PM) to transition from STC89C52 datasheet study to the Week 2 Java Fetch-Decode-Execute CPU implementation.
- **Google Chat & Voice Calls:** Established as the team's primary day-to-day coordination channel during Meeting 2 to resolve early reachability friction from Meeting 1, enabling daily progress checks, rapid voice calls during socket debugging, and asynchronous briefings.
- **Miro Collaborative Board & Shared Google Docs:** Used in Meetings 1 and 2 to partition the 10 STC89C52 hardware study topics across the four members and map out the initial class and process hierarchy before writing code.
- **GitHub Pull Requests, Feature Branches & Decision Logs:** Code was developed across weekly branches (`week-01` through `week-05`) and member branches (`chrisel-CORE`, `arnold-LOG`, `Shashidhara-UI-Final`, `cyrus-integration`) merged via PR #1, with formal minutes archived in `docs/meetings/` and design logs in `docs/decisions/`.

Table 7: Chronological Record of Team Meetings and Communication Channels (docs/meetings/)

Meeting & Date	Channel / Location	Attendance & Duration	Key Technical Decisions & Action Outcomes
Meeting 1 22 Aug 2026	Google Meet (qwi-axoh-wmh) & Miro Board	Cyrus, Chrisel, Shashidhara (Arnold briefed via follow-up)	Partitioned 10 STC89C52 hardware study topics (Memory/PC, Timer/Stack, Interrupts, Registers/PSW/GPIO); initialized GitHub repo.
Meeting 2 27 Aug 2026	Google Meet (Transcribed) & Google Chat	All 4 members present (4:20 PM session)	Finalized reference-code-to-Java simulation workflow (Decision 004); established Google Chat for daily syncs and Miro for task tracking.
Meeting 3 03 Sep 2026	In-Person: College Library	All 4 members present (1:00 PM – 3:30 PM, 2.5 hrs)	Designed Week 2 CPU foundation (Registers, PSW, Memory, logical FETCH->DECODE->EXECUTE stages) and progressive branch testing.
Meeting 4 28 Sep 2026	Google Meet Live Coding & GitHub	Cyrus, Chrisel, Shashidhara (3-hour live session) + Arnold async	Separated simulator into 3 TCP loopback processes: CoreProcess (:5000), LoggerProcess (:5001), and SimulatorUI/UIProcess (:8080).
Meeting 5 03 Nov 2026	Google Meet (hbo-ojhk-fzg)	All 4 members present (10:00 PM – 11:52 PM IST, 1 hr 52 min)	Verified 3GPP CRC-24b (0x800063) across R5:R6:R7, non-blocking ArrayBlockingQueue(1024), branch merges, 38/38 tests, and final report.

6. Problems Faced and How They Were Resolved

Table 8: Engineering Issues Encountered, Root Causes, and Implemented Resolutions

Issue Ref	Category	Symptom / Design Hazard	Root Cause	Implemented Resolution
ISSUE-01	Java Datatypes	Signed integer sign-extension corrupted ADD/SUBB flags	Java lacks unsigned byte (0xFE becomes -2)	Applied bitwise mask (& 0xFF) across all ALU operations and register/RAM reads.
ISSUE-02	Silicon Memory	Stack underflow & stale ROM bytes on reload	SP reset base at 07H [2]; loadProgram needed clear	Guarded pop() at SP <= 0x07 (fix(stack) commit) and zero-filled codeMemory on load.
ISSUE-03	Concurrency	Blocking socket writes can stall CPU if Logger freezes	OS TCP send buffer fills when reader stalls	Bounded ArrayBlockingQueue(1024) with offer() & drop counter (prioritizing CPU determinism).
ISSUE-04	Algorithm	24-bit 3GPP LTE CRC remainder exceeds 8-bit register width	STC89C52 registers are only 8 bits wide	Java routine step8051Hardware() shifts through carry bit across R5:R6:R7 in GF(2) XOR.
ISSUE-05	Fault Handling	DIV AB threw ArithmeticException on divide by zero	Unhandled software division by B=0	Implemented 8051 hardware spec [2]: B=0 sets Overflow (OV=1) without crashing.
ISSUE-06	Web Protocol	Hex bytecode with spaces failed to load from query string	HTTP URL encoding of spaces and symbols	Integrated URLDecoder.decode() in UIProcess /api/cmd HTTP request handler.

7. From Theory to Practice

7.1 Operating Systems Concepts

The project directly applies operating systems principles through process creation, inter-process communication (IPC) via local TCP loopback sockets, and bounded producer-consumer buffering. Decoupling the simulator into discrete OS processes (`CoreProcess`, `LoggerProcess`, `UIProcess`) with explicit monitor synchronization (`cpuLock`, `synchronized LogHandler.writeLog`) and a non-blocking 1,024-slot `ArrayBlockingQueue` ensures that a crash or I/O stall in the UI or disk logger cannot block deterministic CPU instruction execution.

7.2 Data Structures & Algorithms

Core data structures were engineered to model hardware execution buffers: a 16-slot circular FIFO ring buffer with modulo wrap-around pointers (`((rear + 1) % 16)`) verified across 20-item wrap-around bursts, a Last-In First-Out (LIFO) hardware stack starting at `SP=07H` [2] in internal RAM, an opcode dispatch table, and bit-serial GF(2) linear feedback shift register polynomial division across `R5:R6:R7` [5].

7.3 Computer Networks

Computer networking principles govern both the internal socket IPC wire framing and the telecommunications verification lab. The 3GPP TS 36.212 CRC-24b polynomial (g(D) = 0x800063) [5] demonstrates transport block error detection over noisy channels: for payload `"123456789"` (12-byte codeword `31..39 23 EF 52`), clean reception yields syndrome `0x000000`, whereas an exhaustive test sweep verified that all 96 single-bit errors (e.g., byte 0 bit 0 -> `0x520000`) and all 4,560 two-bit error pairs (e.g., byte 0 bits 0&1 -> `0xF60000`) produce non-zero syndromes.

7.4 Cross-Disciplinary Synthesis Reflection

Developing this simulator demonstrated how core computer science disciplines intersect in systems software [4]. Effective virtualization requires balancing computer architecture (instruction-accurate state machines [2]) with operating systems principles (isolated processes, bounded non-blocking queues, synchronization) and networking techniques (TCP framing and GF(2) cyclic error detection [5]).

8. Conclusion, Limitations and Future Work

This project engineered, integrated, and verified an educational multi-process simulator for a 19-opcode subset of the STC89C52 (8051) microcontroller [2]. The system delivers an instruction-accurate Fetch-Decode-Execute CPU engine (supporting both synthetic `0xFF` HALT and standard 8051 `SJMP $` [`80 FE`] halting), 8 KB ROM, 256-byte RAM, a 16-slot circular FIFO queue, an asynchronous append-only TCP logger on Port 5001, a 7-panel Web Dashboard on Port 8080, and a 3GPP TS 36.212 CRC-24b lab [5], passing all 38/38 automated test assertions.

System Scope, Trade-offs & Limitations: (1) The CPU implements 19 representative opcodes (~7.5% of the 255-opcode MCS-51 ISA) and assigns custom educational opcodes `0xE0/0xD0` to FIFO enqueue/dequeue and `0xFF` to synthetic HALT, which collide with standard 8051 `MOVX A,@DPTR`, `POP direct`, and `MOV R7,A`; (2) CRC-24b (`step8051Hardware`) is implemented as a Java routine manipulating the simulated register file (`R5:R6:R7` and `CY`) rather than fetched 8051 machine bytecode; (3) the bounded 1,024-slot log queue prioritizes CPU determinism over log completeness by dropping frames if `LoggerProcess` stalls; and (4) hardware timers and UART interrupts are not yet clocked.

Future Work: Remap the custom FIFO queue and synthetic halt hooks to memory-mapped Special Function Registers (SFRs) or reserved opcode `0xA5`, expand the opcode table to all 255 MCS-51 instructions with machine-cycle timing (e.g., 4-cycle `MUL/DIV`) to run compiled Keil/SDCC Intel HEX binaries, implement CRC-24b in native 8051 assembly, and add CH340 USB-UART hardware-in-the-loop bridging.

References

[1] Department of Computer Science & Engineering, "PBL Simulator Evaluation Rubrics & Task Sheets," St. Aloysius (Deemed to be University), Mangaluru, 2026.

[2] Intel Corporation, "MCS-51 Microcontroller Family User's Manual," Publication No. 272383-002, 1994; and STC Micro, "STC89C52RC/54RD+ Series 8-Bit Microcontroller Datasheet," 2011.

[3] P. C. Vijay Ganesh, "8051 Microcontroller Web-based Simulator," 2021; and J. Rogers, "EdSim51 — The 8051 Simulator for Teachers and Students," Virtualidian, 2019.

[4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2014.

[5] 3GPP TS 36.212, "Evolved Universal Terrestrial Radio Access (E-UTRA); Multiplexing and channel coding (Release 15)," Section 5.1.1 (CRC-24b generator polynomial gCRC24B(D)), 2018.

Final Report Sign-off

By signing below, the team members confirm that the documented contribution distributions, source code statistics, and peer evaluations represent an accurate record of the project lifecycle.

Team Leader Signature: _____ **Date:** 07 November 2026

Faculty Mentor Endorsement:

I have reviewed this Project-Based Learning Final Report along with the associated repository, code contributions, and test suite.

Mentor Signature: _____ **Date:** _____